

Today's Lab — Four Problems on Informed Search

Program 1

8-Puzzle Problem Using Best First Search

Solve the 8-puzzle using Greedy Best First Search. Use the heuristic $h(n)$ = number of misplaced tiles. Generate moves and print the sequence of states from the initial configuration to the goal.

Program 2

A* Algorithm on a Weighted Graph

Implement A* using $f(n) = g(n) + h(n)$ on the same weighted graph from Lab 6. Find the OPTIMAL path from S to G. Print the explored order, optimal path, and total cost.

Program 3

AO* Algorithm on an AND-OR Graph

Implement AO* on the given AND-OR graph. Find the best solution sub-graph by repeatedly choosing the cheapest connector and revising heuristic estimates as nodes are expanded.

Program 4

Run All 5 Algorithms - See Where Each Works and Fails

Run DFS, BFS, Best First and A* on the same weighted graph (Program 2). Then run AO* on the AND-OR graph (Program 3). Compare paths, costs, and discover which properties (completeness, optimality) each satisfies in practice.

The 8-Puzzle You Will Use for Program 1

Initial State

1	2	3
4		6
7	5	8



Goal State

1	2	3
4	5	6
7	8	

Rules and Heuristic

Moves: slide a tile UP, DOWN, LEFT or RIGHT into the blank cell.

State: a 9-element tuple, where 0 marks the blank.

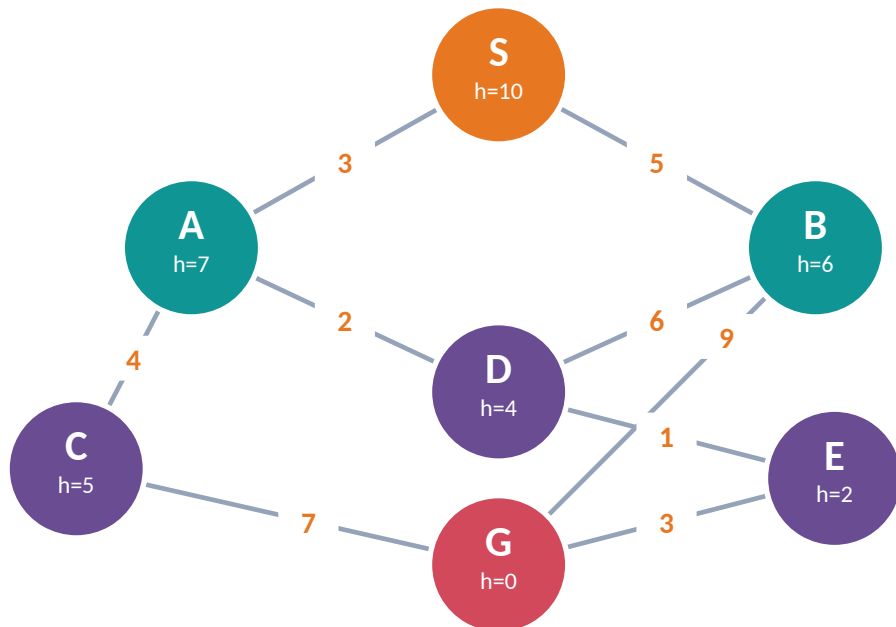
Heuristic $h(n)$: number of MISPLACED tiles (excluding the blank).

For the initial state above:

- tile 5 is at position 7 (should be 4)
- tile 8 is at position 8 (should be 7)

→ $h(\text{initial}) = 2$

The Weighted Graph You Will Use for Program 2 (A*)



Numbers on edges = step cost $g(n)$ h = heuristic estimate to goal G

A* Search Formula

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start S to node n

$h(n)$ = heuristic estimate from n to goal G

$f(n)$ = total estimated cost of path through n

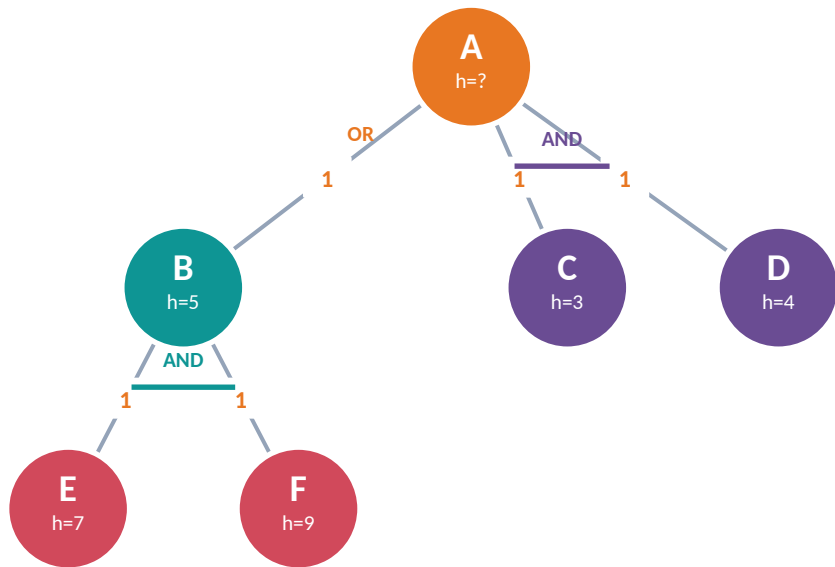
Heuristic values:

$S=10$ $A=7$ $B=6$ $C=5$ $D=4$ $E=2$ $G=0$

Recall — Lab 6 BFS gave cost 14.

A* will find the OPTIMAL cost = 9.

The AND-OR Graph You Will Use for Program 3 (AO*)



All edge costs = 1. Initial heuristics shown inside each node.

AND-OR Graph Rules

OR connector: solve EITHER child to satisfy parent.

AND connector: solve ALL children to satisfy parent.

From A: OR via B, OR AND via (C, D)

From B: AND via (E, F) only

C, D, E, F: terminal (solvable in 0 cost)

AO* picks the cheapest connector at every node, expands it, and REVISES $h(n)$ bottom-up. If a child turns out costlier than estimated, the algorithm may switch to a different connector.

Program 1 — 8-Puzzle Using Best First Search

Program 1

Task: Solve the 8-puzzle using Greedy Best First Search with $h(n)$ = number of misplaced tiles. Print the sequence of moves and states from initial configuration to goal.

Expected Output:

Initial: (1,2,3,4,0,6,7,5,8) $h=2$

Move 1: Down $\rightarrow$ (1,2,3,4,5,6,7,0,8) $h=1$

Move 2: Right $\rightarrow$ (1,2,3,4,5,6,7,8,0) $h=0$ GOAL

Total moves: 2
(0 represents the blank cell)

Steps to implement:

- 1 Define state as a 9-element tuple, 0 = blank
- 2 Define $h(n)$ = count of tiles not in goal position
- 3 Generate successors by sliding blank U/D/L/R
- 4 Use priority queue (min-heap) ordered by $h(n)$
- 5 Print every state and move from start to goal

Program 2 — A* Algorithm on a Weighted Graph

Program 2

Task: Use the A* algorithm with $f(n) = g(n) + h(n)$ to find the OPTIMAL path from S to G on the weighted graph. Print explored order, optimal path, and total cost.

Expected Output:

Explored Order: S A D E G

Optimal Path:

S -> A -> D -> E -> G

Total Cost = 9

(Compare with Lab 6 BFS cost = 14;
A* finds the OPTIMAL path.)

Steps to implement:

- 1 Build graph with edge costs g and heuristics h
- 2 For each node track $g(n)$ and parent pointer
- 3 Use priority queue ordered by $f(n) = g(n) + h(n)$
- 4 If a better g is found later, update and re-push
- 5 Stop when goal is popped; print path and cost

Program 3 — AO* Algorithm on an AND-OR Graph

Program 3

Task: Implement AO* on the given AND-OR graph. At each step pick the cheapest connector, expand it, and revise $h(n)$ bottom-up. Print the solution sub-graph and its total cost.

Expected Output:

```
Pass 1: at A
  OR via B    = 1+5 = 6    <- pick
  AND via C,D = (1+3)+(1+4) = 9
Pass 2: expand B -> AND via E,F = 18,  h(B)=18
Pass 3: revise A -> AND via (C,D) now cheaper
Solution: A -> (C, D)
Total cost = 2      (C, D are leaves, h=0)
```

Steps to implement:

- 1 Represent each node's options as connectors
- 2 Each connector lists its children + arc cost
- 3 Recursively pick the cheapest connector
- 4 Expand chosen node and update its $h(n)$
- 5 Backpropagate h to ancestors; revise marks
- 6 Stop when root is solved; print solution graph

Program 4 - Run All 5 Algorithms; See Where Each Works and Fails

Program 4

Task: Run DFS, BFS, Best First and A* on the weighted graph from Program 2 (S to G). Then run AO* on the AND-OR graph from Program 3, and TRY A* on it too. Tabulate path and cost; observe which algorithms satisfy completeness and optimality in practice.

Expected Output:

Weighted graph (Program 2, S to G):

DFS	S->A->C->G	14	no plan
BFS	S->B->G	14	min edges
Best First	S->B->G	14	greedy fails
A*	S->A->D->E->G	9	OPTIMAL

AND-OR graph (Program 3):

A* fails (no AND). AO*: A->(C,D), cost 2

Steps to implement:

- 1 Run DFS on Program 2 graph; record path and cost
- 2 Run BFS; observe path cost vs A*'s cost
- 3 Run Best First; observe how greedy h misleads it
- 4 Run A*; verify it finds the optimal cost = 9
- 5 Try A* on AND-OR graph; observe it cannot pick AND
- 6 Run AO*; verify A -> (C, D) with cost 2